

LP02 – Récursivité

Définition 1: Récursivité

Un algorithme (ou fonction) récursif est un algorithme (ou fonction) qui fait appel à lui-même.

La principe de la programmation récursive est d'utiliser:

- le cas de base (ou trivial). C'est la condition d'arrêt. (en général c'est une condition facile à trouver car c'est le cas le plus simple).
- une relation de récurrence entre le cas à un instant donné et ce même cas à l'instant suivant.

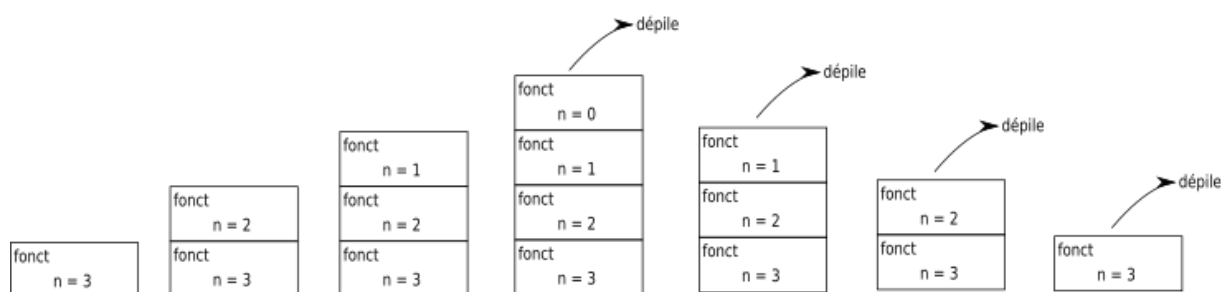
Exemple 1: Les suites récurrentes

Soit la suite (u_n) définie pour tout entier naturel par:

$$\begin{cases} u_0 = 2 \\ u_{n+1} = 2u_n + 1 \end{cases}$$

</> Code Python

```
1 def suite(n):
2     if n == 0:
3         return 2
4     else:
5         return 2 * suite(n-1) + 1
```



appel	n	IF	appel récursif	valeur renvoyée

```
suite(3)
├─ suite(2)
│   └─ suite(1)
│       └─ suite(0) → 2
│           └─ Calcul: 2 × 2 + 1 = 5
│               └─ Calcul: 2 × 5 + 1 = 11
│                   └─ Calcul: 2 × 11 + 1 = 23
```

Exemple 2: La puissance récursive

Soit la fonction définie par:

$$\begin{cases} a^0 = 1 \\ a^{n+1} = a \times a^n \text{ si } n \neq 0 \end{cases}$$

</> Code Python

```
1 def puissance_rec(a, n):
2     if n == 0:
3         return 1
4     else:
5         return a * puissance_rec(a, n-1)
```

Exemple 3: La factorielle récursive

Soit la fonction définie par:

$$\begin{cases} 0! = 1 \\ (n+1)! = (n+1) \times n! \text{ si } n \neq 0 \end{cases}$$

</> Code Python

```
1 def facto_rec(a, n):
2     if n == 0:
3         return 1
4     else:
5         return n * facto_rec(n-1)
```

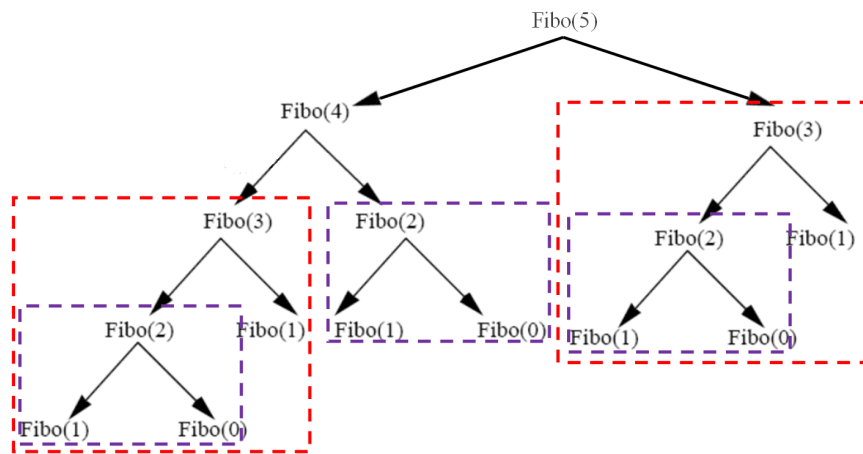
Exemple 4: Suite de Fibonacci récursive

$$\begin{cases} F(0) = 0, \quad F(1) = 1 \\ F(n+2) = F(n+1) + F(n) \end{cases}$$

</> Code Python

```
1 def fibo_rec(n):
2     if n == 0 or n == 1:
3         return n
4     else:
5         return fibo_rec(n-1) + fibo_rec(n-2)
```

Attention, la récursivité n'est pas toujours un gain de complexité:



```

1 -> fibo(5)
  2 -> fibo(4)
    3 -> fibo(3)
      4 -> fibo(2)
        5 -> fibo(1)
          6 -> fibo(0)
      7 -> fibo(1)
    8 -> fibo(2)
      9 -> fibo(1)
        10 -> fibo(0)
  11 -> fibo(3)
    12 -> fibo(2)
      13 -> fibo(1)
        14 -> fibo(0)
    15 -> fibo(1)

```

Exemple 5: D'autres exemples

Somme des éléments dans une liste:

</> Code Python

```

1 def somme(L:list)->int:
2     if L == []:
3         return 0
4     return L.pop() + somme(L)

```

Recherche la présence d'un élément dans une liste

</> Code Python

```

1 def rechercher(L:list, val)->bool:
2     if L == []:
3         return False
4     if L[0] == val:
5         return True
6     return rechercher(L[1:], val)

```

Conversion décimal - binaire

</> Code Python

```

1 def dec2bin(n:int)->str:
2     if n == 0 or n == 1:
3         return str(n)
4     return dec2bin(n//2) + str(n%2)

```

Recherche de la valeur maximale dans une liste

</> Code Python

```

1 def maxi(L:list):
2     if len(L)==1 or L[0]>maxi(L[1:]):
3         return L[0]
4     return maxi(L[1:])

```

Vérifie si une chaîne de caractère est un palindrome

</> Code Python

```

1 def palidrome(mot:str)->bool:
2     if len(mot) <= 1:
3         return True
4     if mot[0] != mot[-1]:
5         return False
6     else:
7         return palidrome(mot[1:-1])

```

Conversion binaire décimale

</> Code Python

```

1 def bin2dec(val:str, i = 0)->int:
2     if len(val) == 1:
3         return int(val)
4     return int(val[-1])*2**i + \
5         bin2dec(val[:-1], i+1)

```